Celebal Summer Internship 2026

Week 2 Task: E-Commerce Sales Database

1. Scenario & Business Context

You are a Junior Data Analyst at ShopEase, a mid-sized e-commerce company that sells electronics, clothing, and home products across India. The management team wants to understand sales patterns, customer behavior, and product performance to make better business decisions.

You have been given access to the company's relational database containing information about customers, products, orders, and order details. Your task is to write SQL queries to extract meaningful insights from this data.

⚠ Note: You may use MySQL, PostgreSQL, or any SQL-compatible RDBMS to complete these tasks. All queries should follow standard SQL syntax.

2. Database Schema & Constraints

The database consists of 4 tables. Study the schema below carefully — pay attention to Primary Keys, Foreign Keys, and Constraints.

Table: customers

```
CREATE TABLE customers (
    customer_id   INT           PRIMARY KEY,
    first_name    VARCHAR(50)   NOT NULL,
    last_name     VARCHAR(50)   NOT NULL,
    email         VARCHAR(100)  UNIQUE NOT NULL,
    city          VARCHAR(50)   NOT NULL,
    state         VARCHAR(50)   NOT NULL,
    join_date     DATE          NOT NULL,
    is_premium    BOOLEAN       DEFAULT FALSE
);

-- Index for filtering by city/state
CREATE INDEX idx_customers_city ON customers(city);
CREATE INDEX idx_customers_state ON customers(state);
```

Table: products

```
CREATE TABLE products (
    product_id    INT           PRIMARY KEY,
    product_name  VARCHAR(100)  NOT NULL,
    category      VARCHAR(50)   NOT NULL,
    brand         VARCHAR(50)   NOT NULL,
    unit_price    DECIMAL(10,2) NOT NULL  CHECK (unit_price > 0),
    stock_qty     INT           NOT NULL  DEFAULT 0  CHECK (stock_qty >= 0)
);

-- Index for filtering by category
CREATE INDEX idx_products_category ON products(category);
```

Table: orders

```
CREATE TABLE orders (
    order_id      INT           PRIMARY KEY,
    customer_id   INT           NOT NULL,
    order_date    DATE          NOT NULL,
    status        VARCHAR(20)   NOT NULL  DEFAULT 'Pending'
```

```
                  CHECK (status IN ('Pending','Shipped','Delivered','Cancelled')),
    total_amount  DECIMAL(12,2) NOT NULL  CHECK (total_amount >= 0),

    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

-- Index for date-based filtering and sorting
CREATE INDEX idx_orders_date ON orders(order_date);
CREATE INDEX idx_orders_status ON orders(status);
```

Table: order_items

```
CREATE TABLE order_items (
    item_id       INT           PRIMARY KEY,
    order_id      INT           NOT NULL,
    product_id    INT           NOT NULL,
    quantity      INT           NOT NULL  CHECK (quantity > 0),
    unit_price    DECIMAL(10,2) NOT NULL  CHECK (unit_price > 0),
    discount_pct  DECIMAL(5,2)  DEFAULT 0 CHECK (discount_pct BETWEEN 0 AND 100),

    FOREIGN KEY (order_id)   REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

Entity Relationships

```
  customers  ──(1:N)──▶  orders
  orders     ──(1:N)──▶  order_items
  products   ──(1:N)──▶  order_items

  customers.customer_id  ◀──FK──  orders.customer_id
  orders.order_id        ◀──FK──  order_items.order_id
  products.product_id    ◀──FK──  order_items.product_id
```

3. Sample Data

Below is sample data to help you understand the tables. Use the provided INSERT statements to load data into your database.

customers (sample rows)

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	TRUE
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	FALSE
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	TRUE
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	FALSE
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	TRUE
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	FALSE
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	TRUE
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	FALSE

products (sample rows)

product_id	product_name	category	brand	unit_price	stock_qty
201	Wireless Earbuds	Electronics	BoAt	1499.0	250
202	Cotton T-Shirt	Clothing	Levi's	799.0	500
203	Smart Watch	Electronics	Noise	2999.0	150
204	Running Shoes	Clothing	Nike	4599.0	120
205	Bluetooth Speaker	Electronics	JBL	3499.0	200
206	Bedsheet Set	Home	Spaces	1299.0	300
207	Laptop Stand	Electronics	AmazonBasics	899.0	180
208	Cushion Covers (Set)	Home	HomeCenter	599.0	400

orders (sample rows)

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.0
1002	102	2024-08-03	Delivered	799.0
1003	103	2024-08-05	Shipped	7498.0
1004	101	2024-08-10	Delivered	3499.0
1005	104	2024-08-12	Cancelled	2999.0
1006	105	2024-08-15	Delivered	5898.0
1007	106	2024-08-18	Pending	1299.0
1008	103	2024-08-20	Delivered	899.0
1009	107	2024-08-25	Shipped	6098.0
1010	108	2024-08-28	Delivered	1598.0

order_items (sample rows)

item_id	order_id	product_id	quantity	unit_price	discount_pct
5001	1001	201	2	1499.0	0
5002	1001	207	1	899.0	10
5003	1002	202	1	799.0	0
5004	1003	203	1	2999.0	0
5005	1003	204	1	4599.0	5
5006	1004	205	1	3499.0	0
5007	1005	203	1	2999.0	0
5008	1006	201	1	1499.0	10
5009	1006	204	1	4599.0	5
5010	1007	206	1	1299.0	0
5011	1008	207	1	899.0	0
5012	1009	205	1	3499.0	0
5013	1009	208	2	599.0	15

5014	1010	206	1	1299.0	0
5015	1010	208	1	599.0	0

Data Loading — INSERT Statements

Copy and execute the following SQL to load sample data into your database:

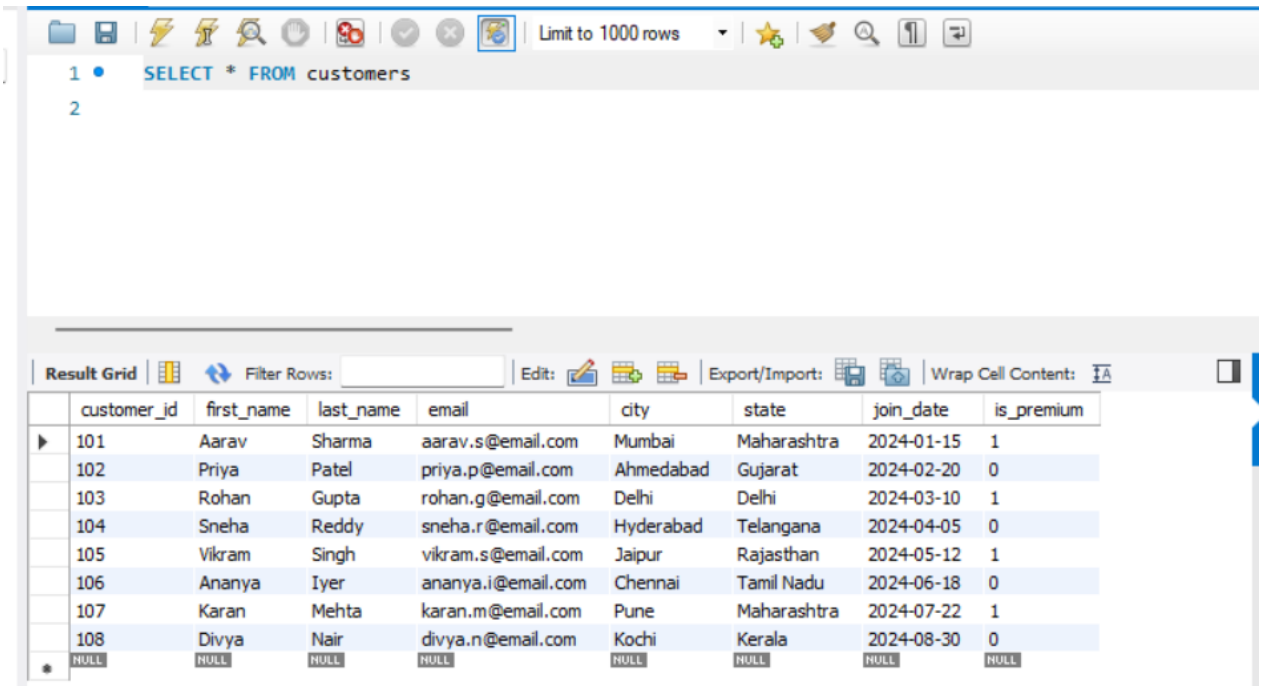
```
-- ========== INSERT: customers ==========
INSERT INTO customers VALUES
(101, 'Aarav',  'Sharma', 'aarav.s@email.com',  'Mumbai',    'Maharashtra', '2024-01-15',
TRUE),
(102, 'Priya',  'Patel',  'priya.p@email.com',  'Ahmedabad', 'Gujarat',     '2024-02-20',
FALSE),
(103, 'Rohan',  'Gupta',  'rohan.g@email.com',  'Delhi',     'Delhi',       '2024-03-10',
TRUE),
(104, 'Sneha',  'Reddy',  'sneha.r@email.com',  'Hyderabad', 'Telangana',   '2024-04-05',
FALSE),
(105, 'Vikram', 'Singh',  'vikram.s@email.com', 'Jaipur',    'Rajasthan',   '2024-05-12',
TRUE),
(106, 'Ananya', 'Iyer',   'ananya.i@email.com', 'Chennai',   'Tamil Nadu',  '2024-06-18',
FALSE),
(107, 'Karan',  'Mehta',  'karan.m@email.com',  'Pune',      'Maharashtra', '2024-07-22',
TRUE),
(108, 'Divya',  'Nair',   'divya.n@email.com',  'Kochi',     'Kerala',      '2024-08-30',
FALSE);

-- ========== INSERT: products ==========
INSERT INTO products VALUES
(201, 'Wireless Earbuds',     'Electronics', 'BoAt',          1499.00, 250),
(202, 'Cotton T-Shirt',       'Clothing',    'Levis',         799.00,  500),
(203, 'Smart Watch',          'Electronics', 'Noise',         2999.00, 150),
(204, 'Running Shoes',        'Clothing',    'Nike',          4599.00, 120),
(205, 'Bluetooth Speaker',    'Electronics', 'JBL',           3499.00, 200),
(206, 'Bedsheet Set',         'Home',        'Spaces',        1299.00, 300),
(207, 'Laptop Stand',         'Electronics', 'AmazonBasics',  899.00,  180),
(208, 'Cushion Covers (Set)', 'Home',        'HomeCenter',    599.00,  400);

-- ========== INSERT: orders ==========
INSERT INTO orders VALUES
(1001, 101, '2024-08-01', 'Delivered',  4498.00),
(1002, 102, '2024-08-03', 'Delivered',  799.00),
(1003, 103, '2024-08-05', 'Shipped',    7498.00),
(1004, 101, '2024-08-10', 'Delivered',  3499.00),
(1005, 104, '2024-08-12', 'Cancelled',  2999.00),
(1006, 105, '2024-08-15', 'Delivered',  5898.00),
(1007, 106, '2024-08-18', 'Pending',    1299.00),
(1008, 103, '2024-08-20', 'Delivered',  899.00),
(1009, 107, '2024-08-25', 'Shipped',    6098.00),
(1010, 108, '2024-08-28', 'Delivered',  1598.00);

-- ========== INSERT: order_items ==========
INSERT INTO order_items VALUES
(5001, 1001, 201, 2, 1499.00, 0),
(5002, 1001, 207, 1, 899.00,  10),
(5003, 1002, 202, 1, 799.00,  0),
(5004, 1003, 203, 1, 2999.00, 0),
(5005, 1003, 204, 1, 4599.00, 5),
(5006, 1004, 205, 1, 3499.00, 0),
```

```
(5007, 1005, 203, 1, 2999.00, 0),
(5008, 1006, 201, 1, 1499.00, 10),
(5009, 1006, 204, 1, 4599.00, 5),
(5010, 1007, 206, 1, 1299.00, 0),
(5011, 1008, 207, 1, 899.00,  0),
(5012, 1009, 205, 1, 3499.00, 0),
(5013, 1009, 208, 2, 599.00,  15),
(5014, 1010, 206, 1, 1299.00, 0),
(5015, 1010, 208, 1, 599.00,  0);
```

Section A — SQL Basics (SELECT, Constraints, Primary Keys)

These questions test your understanding of basic data retrieval, table structure, and database constraints.

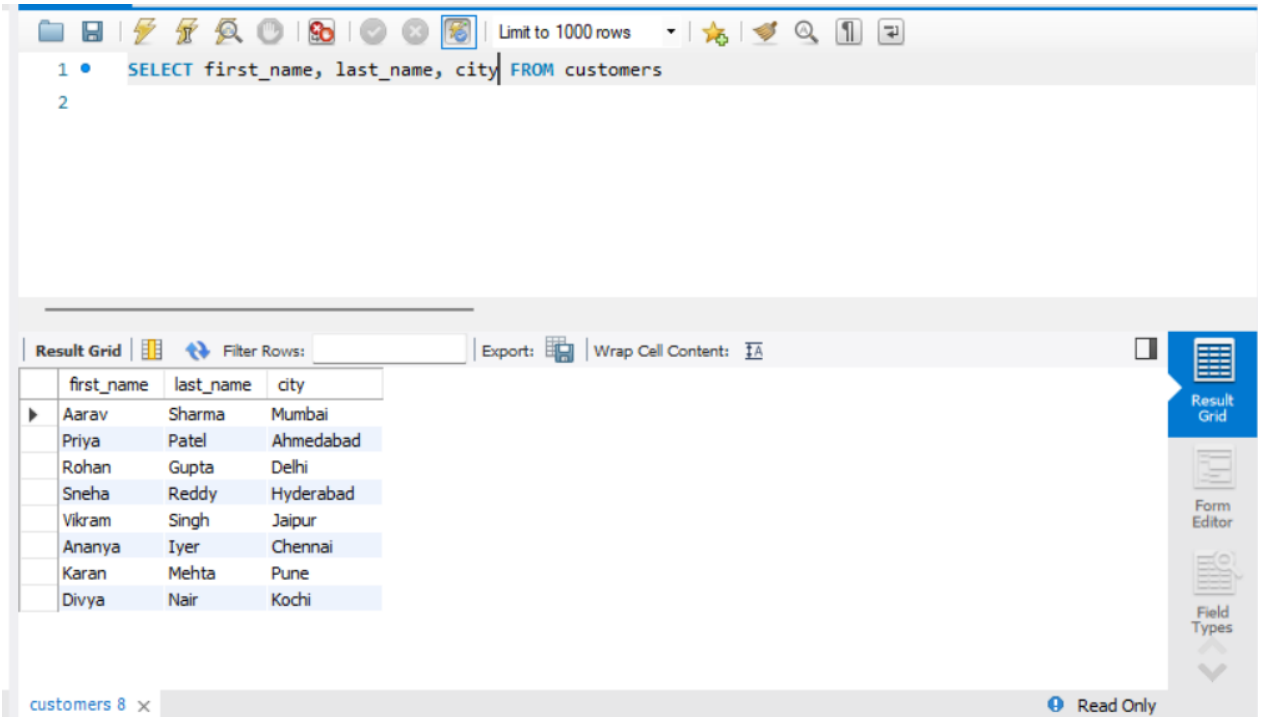
Q1. Write a query to display all columns and rows from the customer's table.



```
SELECT * FROM customers
```

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0

Q2. Retrieve only the first_name, last_name, and city of all customers.



```
SELECT first_name, last_name, city FROM customers
```

first_name	last_name	city
Aarav	Sharma	Mumbai
Priya	Patel	Ahmedabad
Rohan	Gupta	Delhi
Sneha	Reddy	Hyderabad
Vikram	Singh	Jaipur
Ananya	Iyer	Chennai
Karan	Mehta	Pune
Divya	Nair	Kochi

Q3. List all unique categories available in the products table.

```
1 •  SELECT DISTINCT category
2    FROM products;
3
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category
Clothing
Electronics
Home

Result Grid | Form Editor | Field Types

Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

- customers → customer_id
- products → product_id
- orders → order_id
- order_items → item_id

PRIMARY KEY :- A primary key is used to uniquely identify each record in a table. It must be **UNIQUE** so that no two rows have the same value and it cannot be **NULL** because every record should have its own valid identifier. This helps in maintains relationships between tables.

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

The email column has UNIQUE and NOT NULL constraints. If a duplicate email is inserted, MySQL returns a duplicate entry error and the record is not inserted

Limit to 1000 rows

```
1 •  INSERT INTO customers VALUES
2    (109, 'Manish',  'Khyalia', 'aarav.s@email.com',  'Sikar','Rajasthan', '2026-06-25', TRUE);
3
```

25 13:58:20 INSERT INTO customers VALUES (109, 'Manish', 'Khyalia', 'aarav.s@email.com', 'Sikar','Rajasthan', '2026-0... Error Code: 1062. Duplicate entry 'aarav.s@email.com' for key 'customers.email' 0.016 sec

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

The query fails because the CHECK (unit_price > 0) constraint does not allow negative prices.


1 INSERT INTO products VALUES
2 (209, 'Wireless Earbuds', 'Electronics', 'BoAt', -50, 250);
3
26 14:03:18 INSERT INTO products VALUES (209, 'Wireless Earbuds', 'Electronics', 'BoAt', -50, 250) Error Code: 3819. Check constraint 'products_chk_1' is violated. 0.000 sec

Section B — Filtering & Optimization (WHERE, Indexes)

These questions test your ability to filter data effectively and understand how indexes improve query performance.

Q7. Retrieve all orders with status = 'Delivered'.

```
1 •  SELECT *
2    FROM orders
3    WHERE status = 'Delivered';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.00
1002	102	2024-08-03	Delivered	799.00
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1008	103	2024-08-20	Delivered	899.00
1010	108	2024-08-28	Delivered	1598.00
NULL	NULL	NULL	NULL	NULL

orders 10 | Apply | Revert

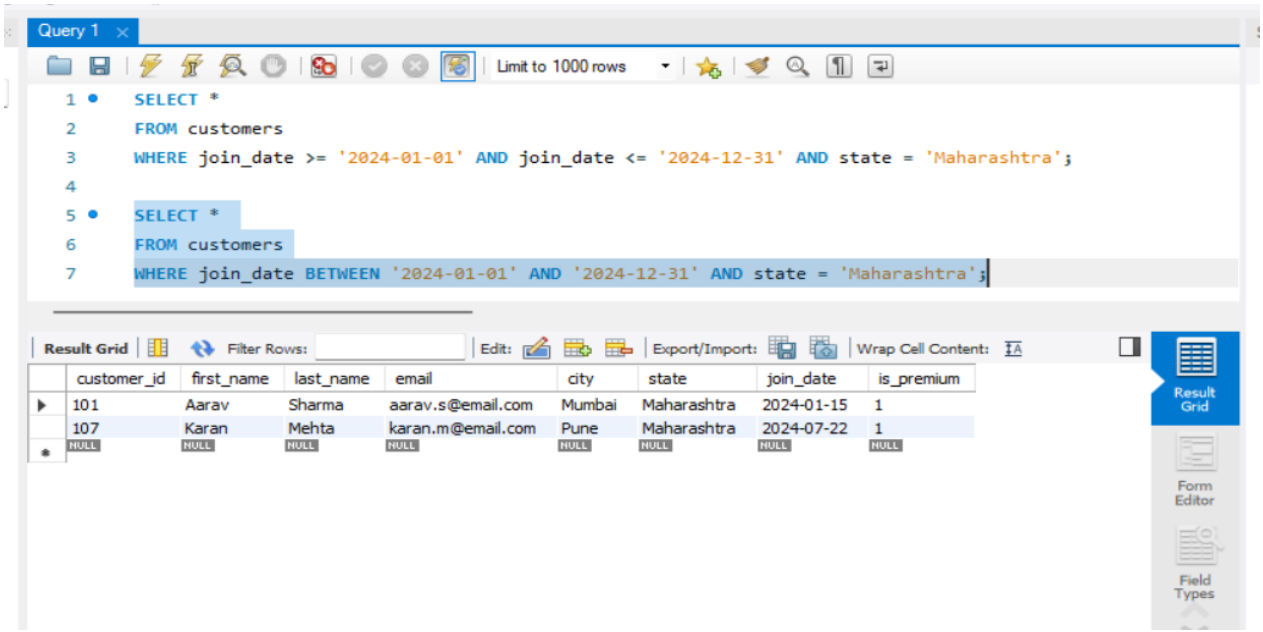
Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

```
1 •  SELECT *
2    FROM products
3    WHERE category = 'Electronics' AND unit_price > 2000;
```

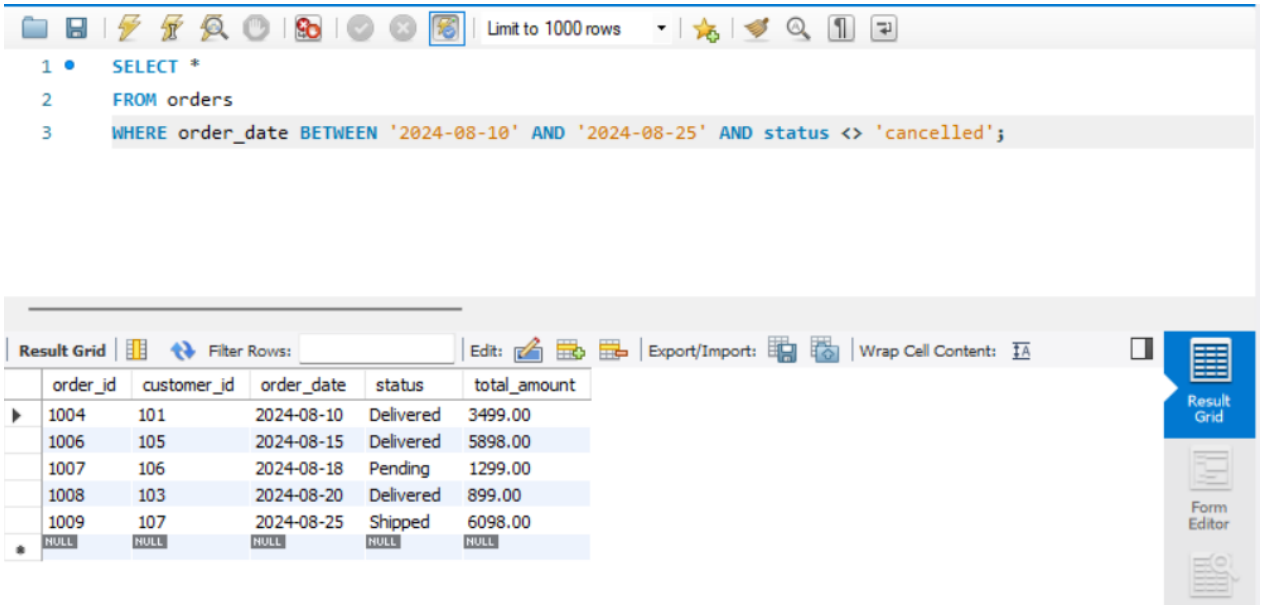
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

product_id	product_name	category	brand	unit_price	stock_qty
203	Smart Watch	Electronics	Noise	2999.00	150
205	Bluetooth Speaker	Electronics	JBL	3499.00	200
NULL	NULL	NULL	NULL	NULL	NULL

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.



Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.



Q11. Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a sample query that would benefit from this index.

Query 1

Limit to 1000 rows

```
1  SELECT *
2  FROM orders
3  WHERE order_date = '2024-08-10';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
NULL	NULL	NULL	NULL	NULL

Result Grid

The idx_orders_date index is created on the order_date column. It helps MySQL find orders by date more quickly instead of checking every row in the table. This improves the performance of queries that filter records using order_date.

Q12. If you run: SELECT * FROM customers WHERE YEAR(join_date) = 2024; — would the index on join_date be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

No, the index is not used because the YEAR() function is applied to the join_date column. It is better to use a date range so MySQL can use the index and find the data faster.

Limit to 1000 rows

```
1  SELECT *
2  FROM customers
3  WHERE YEAR(join_date) = 2024;
```

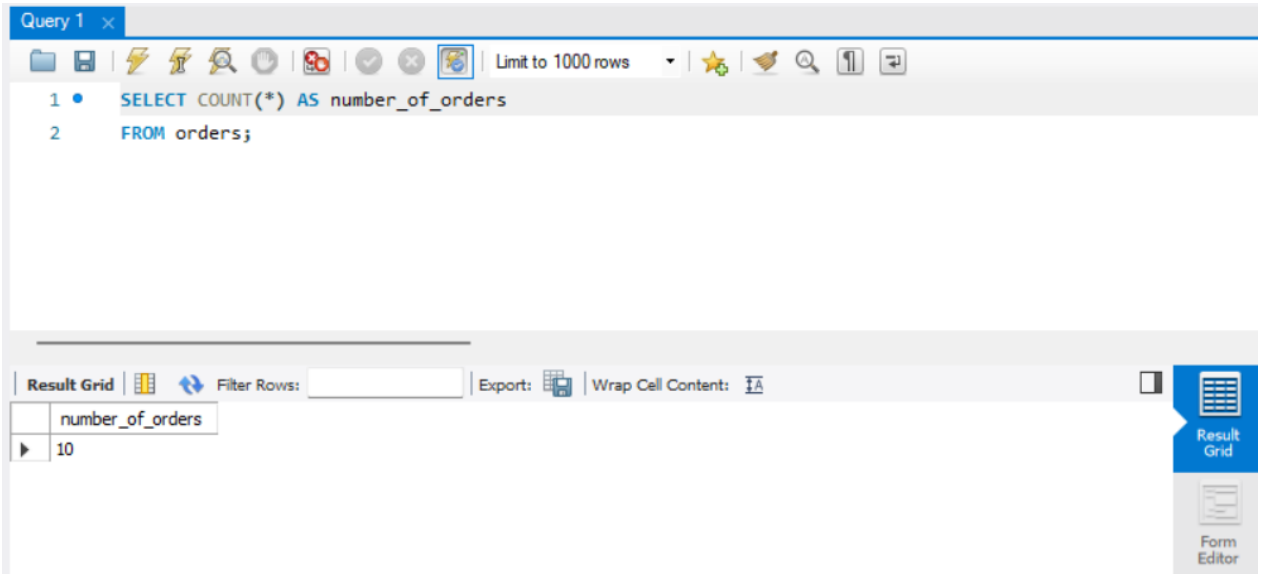
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

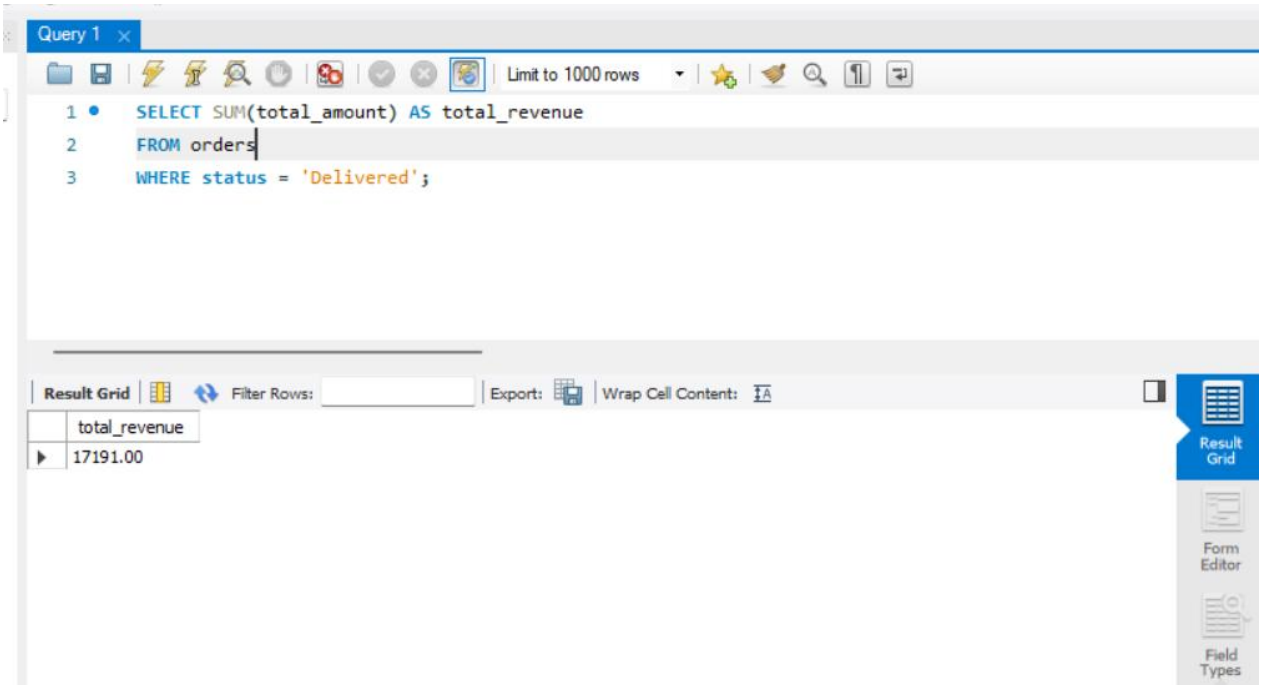
Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

These questions test your ability to summarize and aggregate data.

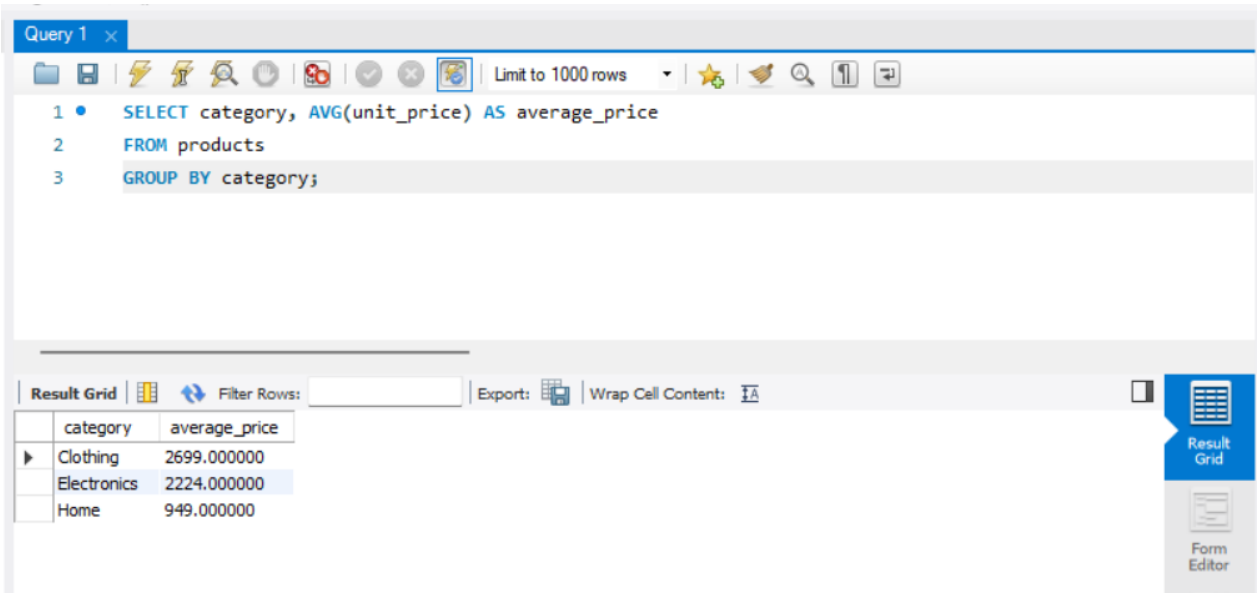
Q13. Count the total number of orders in the orders table.



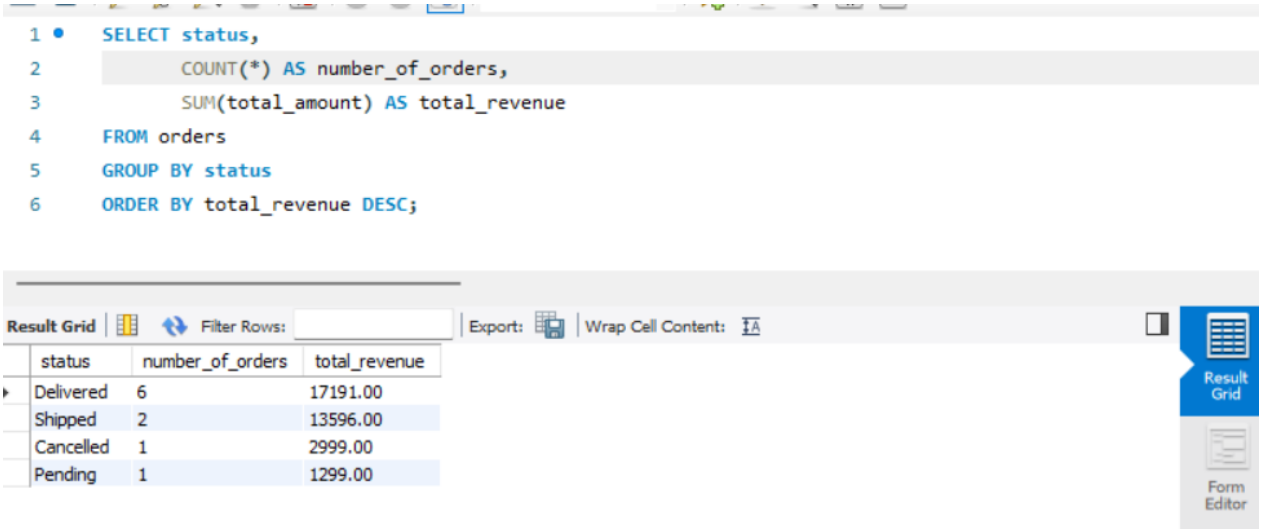
Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.



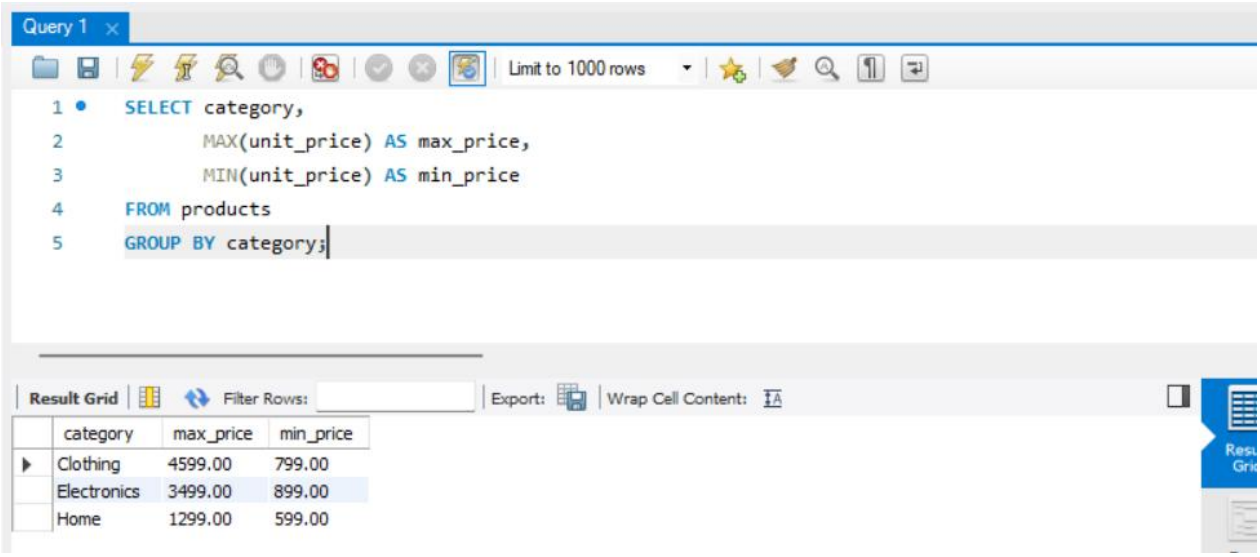
Q15. Calculate the average unit_price of products in each category.



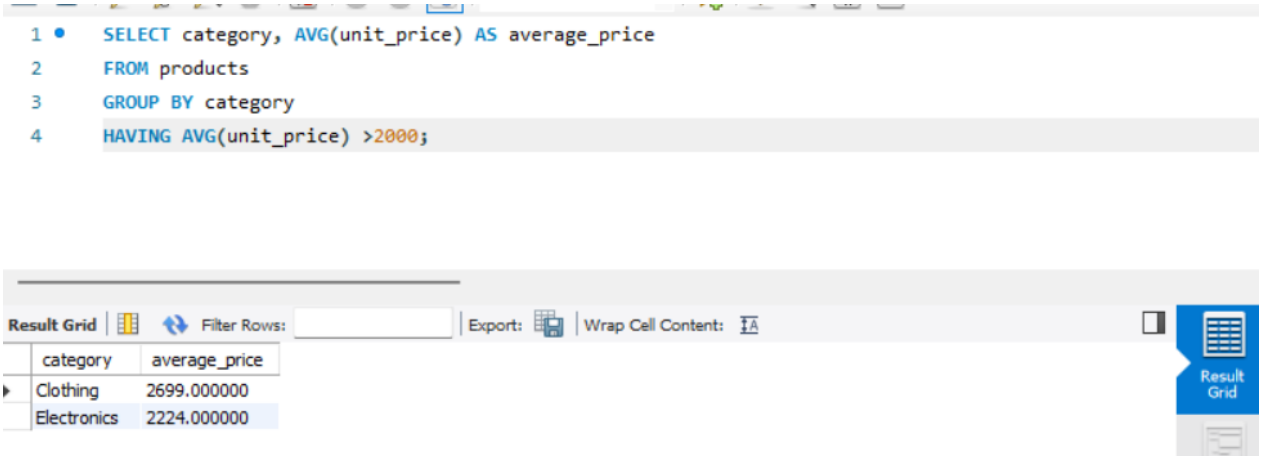
Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.



Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.



Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)



Section D — Joins & Relationships

These questions test your ability to combine data from multiple tables using different types of JOINs.

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.

```sql
SELECT o.order_id, o.order_date,
       c.first_name, c.last_name, o.total_amount
FROM orders AS o
INNER JOIN customers AS c
ON o.customer_id = c.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498.00
1004	2024-08-10	Aarav	Sharma	3499.00
1002	2024-08-03	Priya	Patel	799.00
1011	2026-06-28	Priya	Patel	1598.00
1003	2024-08-05	Rohan	Gupta	7498.00
1008	2024-08-20	Rohan	Gupta	899.00
1005	2024-08-12	Sneha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5898.00
1007	2024-08-18	Ananya	Iyer	1299.00
1009	2024-08-25	Karan	Mehta	6098.00
1010	2024-08-28	Divya	Nair	1598.00

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

```sql
SELECT c.customer_id, c.first_name, c.last_name,
       o.order_id, o.order_date, o.total_amount
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	first_name	last_name	order_id	order_date	total_amount
101	Aarav	Sharma	1001	2024-08-01	4498.00
101	Aarav	Sharma	1004	2024-08-10	3499.00
102	Priya	Patel	1002	2024-08-03	799.00
103	Rohan	Gupta	1003	2024-08-05	7498.00
103	Rohan	Gupta	1008	2024-08-20	899.00
104	Sneha	Reddy	1005	2024-08-12	2999.00
105	Vikram	Singh	1006	2024-08-15	5898.00
106	Ananya	Iyer	1007	2024-08-18	1299.00
107	Karan	Mehta	1009	2024-08-25	6098.00
108	Divya	Nair	1010	2024-08-28	1598.00

Result Grid | Form Editor | Field Types

Q21. Write a query using JOINs across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

```sql
SELECT o.order_id, p.product_name, oi.quantity, oi.unit_price, oi.discount_pct
FROM orders o
INNER JOIN order_items oi
ON o.order_id = oi.order_id
INNER JOIN products p
ON oi.product_id = p.product_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	product_name	quantity	unit_price	discount_pct
1011	Wireless Earbuds	1	999.00	0.00
1001	Wireless Earbuds	2	1499.00	0.00
1006	Wireless Earbuds	1	1499.00	10.00
1011	Cotton T-Shirt	1	599.00	0.00
1002	Cotton T-Shirt	1	799.00	0.00
1003	Smart Watch	1	2999.00	0.00
1005	Smart Watch	1	2999.00	0.00
1003	Running Shoes	1	4599.00	5.00
1006	Running Shoes	1	4599.00	5.00
1004	Bluetooth Speaker	1	3499.00	0.00
1009	Bluetooth Speaker	1	3499.00	0.00
1007	Bedsheet Set	1	1299.00	0.00
1010	Bedsheet Set	1	1299.00	0.00
1001	Laptop Stand	1	899.00	10.00
1008	Laptop Stand	1	899.00	0.00
1009	Cushion Covers (...	2	599.00	15.00
1010	Cushion Covers (...	1	599.00	0.00

Result 33 | Read Only

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

LEFT JOIN: Returns all records from the left table and the matching records from the right table. If there is no match, the right table columns show NULL.

RIGHT JOIN: Returns all records from the right table and the matching records from the left table. If there is no match, the left table columns show NULL.

Syntax :

```sql
-- LEFT JOIN Example:
SELECT *
FROM customers
LEFT JOIN orders
ON customers.customer_id = orders.customer_id;
```

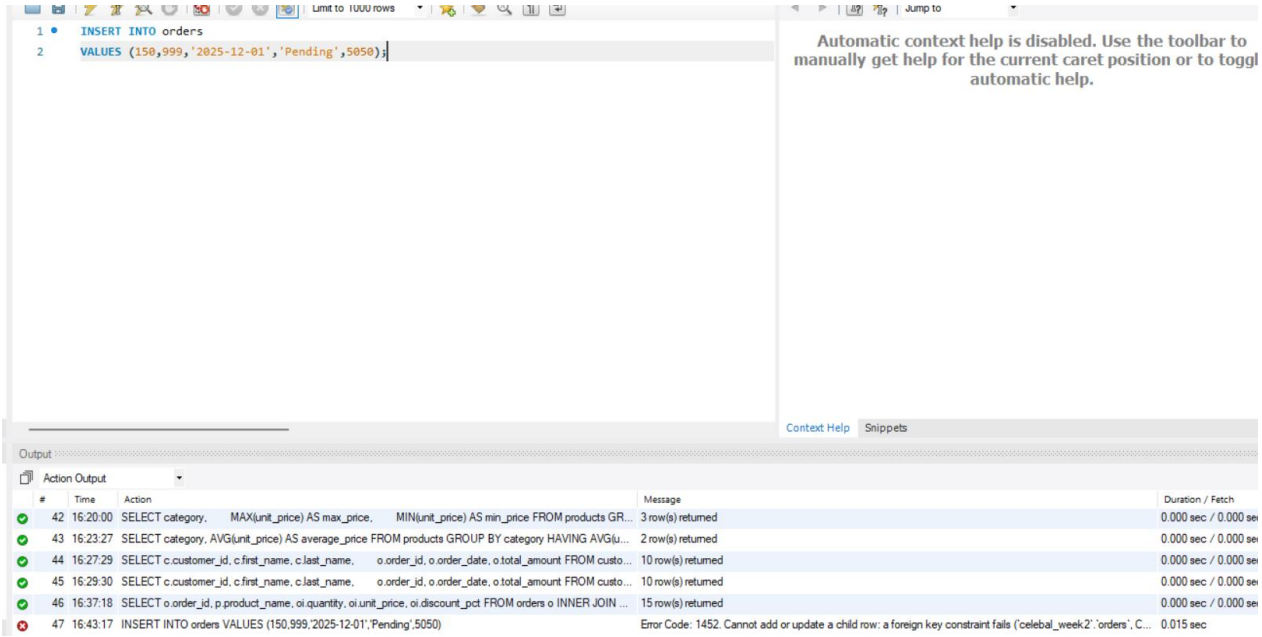
```sql
-- RIGHT JOIN Example:
SELECT *
FROM customers
RIGHT JOIN orders
ON customers.customer_id = orders.customer_id;
```

FULL OUTER JOIN: It returns all records from both tables, whether they match or not. It is useful when you want to see every customer and every order, including records that have no matching row.

Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

- orders.customer_id → customers.customer_id
- order_items.order_id → orders.order_id
- order_items.product_id → products.product_id

If I try to insert an order with customer_id = 999, the query will fail because this customer does not exist in the customers table. The foreign key constraint prevents invalid data from being inserted and keeps the relationship between the tables correct.



Section E — Advanced Concepts (CASE, ACID, Transactions)

These questions test your understanding of conditional logic, database reliability principles, and transaction management.

Q24. Write a query using CASE to classify products into price tiers:

- 'Budget' → unit_price < 1000
- 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
- 'Premium' → unit_price > 3000

Display: product_name, unit_price, price_tier.

```sql
SELECT product_name, unit_price,
       CASE
           WHEN unit_price < 1000 THEN 'Budget'
           WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
           ELSE 'Premium'
       END AS price_tier
FROM products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	unit_price	price_tier
Wireless Earbuds	1499.00	Mid-Range
Cotton T-Shirt	799.00	Budget
Smart Watch	2999.00	Mid-Range
Running Shoes	4599.00	Premium
Bluetooth Speaker	3499.00	Premium
Bedsheet Set	1299.00	Mid-Range
Laptop Stand	899.00	Budget
Cushion Covers (Set)	599.00	Budget

Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

```sql
SELECT
    SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS Delivered,
    SUM(CASE WHEN status <> 'Delivered' THEN 1 ELSE 0 END) AS Not_Delivered
FROM orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Delivered	Not_Delivered
6	4

Q26. Explain each letter of ACID:

- A – Atomicity
- C – Consistency
- I – Isolation
- D – Durability

Give a real-world example (e.g., bank transfer) showing why each property is important.

A – Atomicity:
A transaction is either completed fully or not completed at all.
Example: If money is deducted from one account but cannot be added to another, the transaction is cancelled.

C – Consistency:
The database always stays correct before and after a transaction.
Example: The total amount of money remains the same after a bank transfer.

I – Isolation:
Multiple transactions can run at the same time without affecting each other.
Example: Two people transferring money at the same time do not interfere with each other's transactions.

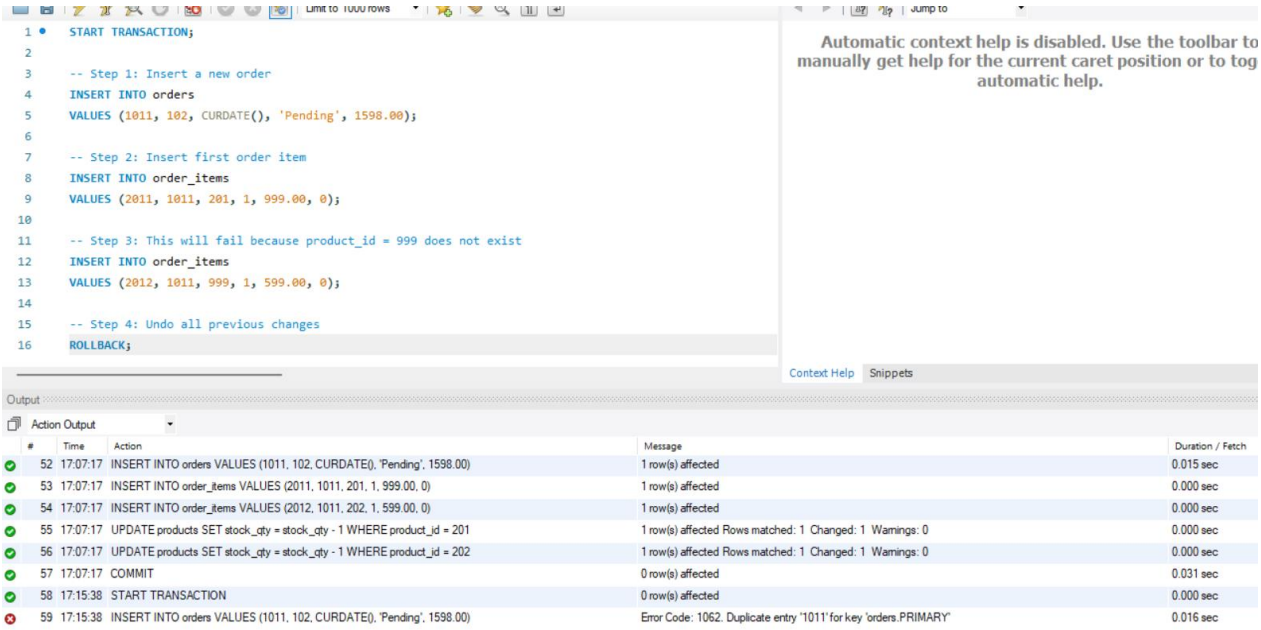
D – Durability:
Once a transaction is committed, the changes are saved permanently.
Example: After a successful bank transfer, the transaction remains saved even if the system crashes.

Q27. Write a SQL transaction that does the following atomically:

1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)
2. Insert two order items for that order
3. Update the stock_qty of the purchased products
4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.

Write the complete BEGIN...COMMIT/ROLLBACK block.

```sql
START TRANSACTION;
-- 1. Insert a new order
INSERT INTO orders
VALUES (1011, 102, CURDATE(), 'Pending', 1598.00);

-- 2. Insert two order items
INSERT INTO order_items
VALUES (2011, 1011, 201, 1, 999.00, 0);
INSERT INTO order_items
VALUES (2012, 1011, 202, 1, 599.00, 0);

-- 3. Update the stock of purchased products
UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 201;
UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 202;

-- 4. If all statements are successful
COMMIT;

```



```
1  START TRANSACTION;
2
3  -- Step 1: Insert a new order
4  INSERT INTO orders
5  VALUES (1011, 102, CURDATE(), 'Pending', 1598.00);
6
7  -- Step 2: Insert first order item
8  INSERT INTO order_items
9  VALUES (2011, 1011, 201, 1, 999.00, 0);
10
11 -- Step 3: This will fail because product_id = 999 does not exist
12 INSERT INTO order_items
13 VALUES (2012, 1011, 999, 1, 599.00, 0);
14
15 -- Step 4: Undo all previous changes
16 ROLLBACK;
```

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to tog automatic help.

Context Help | Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
52	17:07:17	INSERT INTO orders VALUES (1011, 102, CURDATE(), 'Pending', 1598.00)	1 row(s) affected	0.015 sec
53	17:07:17	INSERT INTO order_items VALUES (2011, 1011, 201, 1, 999.00, 0)	1 row(s) affected	0.000 sec
54	17:07:17	INSERT INTO order_items VALUES (2012, 1011, 202, 1, 599.00, 0)	1 row(s) affected	0.000 sec
55	17:07:17	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 201	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
56	17:07:17	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 202	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
57	17:07:17	COMMIT	0 row(s) affected	0.031 sec
58	17:15:38	START TRANSACTION	0 row(s) affected	0.000 sec
59	17:15:38	INSERT INTO orders VALUES (1011, 102, CURDATE(), 'Pending', 1598.00)	Error Code: 1062. Duplicate entry '1011' for key 'orders.PRIMARY'	0.016 sec

If all statements execute successfully, the transaction is committed. If any statement fails, all changes are rolled back.

----------END----------